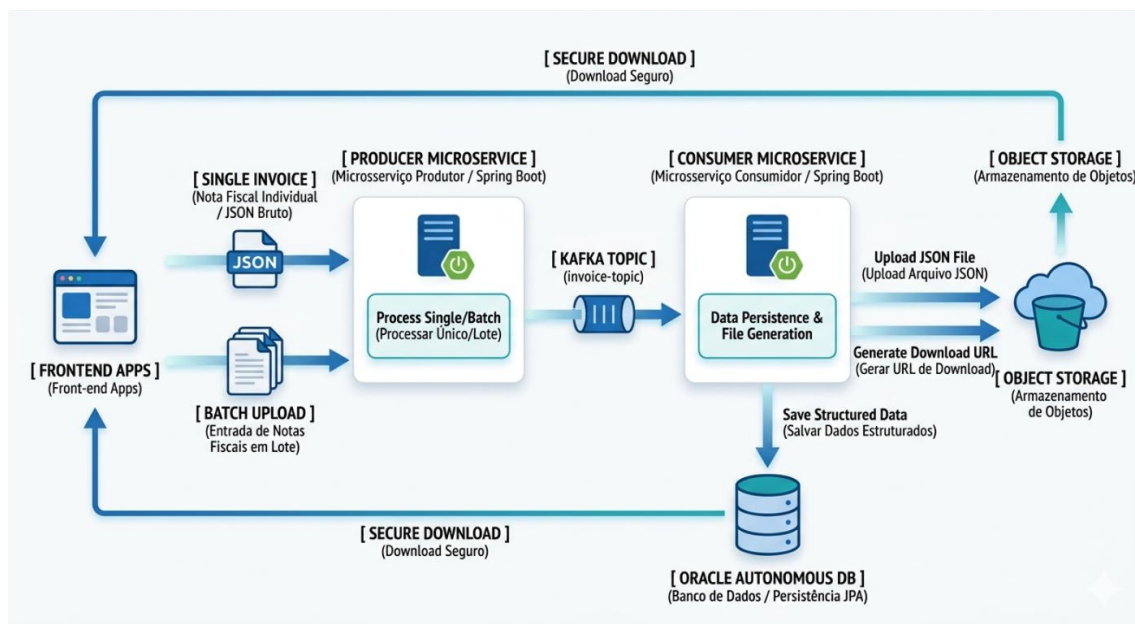


Cloud-Native Asynchronous Invoice Processing System

Project Overview

This project consists of a highly scalable, fault-tolerant platform for processing, persisting, and archiving Electronic Invoices. Developed under a **Decoupled Microservices** architecture, the system leverages asynchronous messaging, container orchestration within a hybrid Kubernetes cluster, automated security, and dual-layer persistence (relational and object storage) using capabilities of the **Oracle Cloud Infrastructure (OCI)** ecosystem.



"This diagram illustrates the **End-to-End Data Pipeline Lifecycle** implemented in the platform. The architecture is split into 4 fully decoupled macro-steps:

1. **Reactive Ingestion (Edge)**: The client-side dashboard dispatches structured payloads as Single Invoices or massive Batch loads in JSON format to the **Producer Microservice (8081)**.
2. **Messaging & Decoupling**: The Producer validates the payload and instantly publishes the event to the **Apache Kafka Topic** running in **KRaft mode**, freeing up the UI immediately and securing overall system fault tolerance.
3. **Asynchronous Hybrid Processing**: The **Consumer Microservice (8082)** intercepts the message from the Kafka grid and splits the execution into two parallel branches: it persists structured data into the **Oracle Autonomous Database** via JPA and handles the background physical upload of the .json file into the **OCI Object Storage** based in São Paulo.
4. **Secure Download Tunneling (PAR)**: Whenever the client layer requests an audit for a document or a complete batch, the Consumer triggers a request to generate temporary cryptographic keys straight on the Oracle Object Storage.

This returns a clean *Blob Streaming* flow (or packaged .zip compression files) directly to the user's OS without ever exposing the master credentials of the cloud architecture.

1. Development Architecture & Microservices (Spring Boot)

The core business logic was developed using **Java 17** and **Spring Boot 3**, focusing on Clean Architecture and isolated responsibilities:

- **Invoice Producer Microservice (API Gateway):**
 - Acts as the entry point of the data flow (Edge Service), exposed on port 8081.
 - Provides REST controllers prepared for two distinct high-load scenarios: ingestion of **Raw Single JSON** payloads (/single) or asynchronous processing of **Invoice Batches** (/batch).
 - Handles data contract conversion via **DTOs (Data Transfer Objects)** and immediately publishes events to the messaging broker.
- **Invoice Consumer Microservice (API web & Worker Listener):**
 - A pure asynchronous background worker (spring.main.web-application-type=none) optimized for performance, eliminating Tomcat server overhead to save RAM.
 - Ensures absolute resilience during data deserialization using a **Raw String + Jackson Object Mapper** strategy, preventing infrastructure crashes caused by malformed payloads (*Poison Pills*).
 - Guarantees absolute resilience during deserialization by adopting a strategy based on raw Strings + Jackson ObjectMapper, shielding the infrastructure against failures caused by invalid payloads (**Poison Pills**).
 - Hosts the **InvoiceHistoryController** (managing and automating the invoice history hydration from the database on page refresh / F5) and the **DownloadController** (native integration with the Oracle Cloud SDK for generating temporary, cryptographically secure **PAR - Pre-Authenticated Request** links).
- **Advanced Persistence (JPA / Hibernate):**
 - Implementation of **Spring Data JPA** wrapping **Hibernate** as the underlying object-relational mapping engine.
 - An isolated **Service** layer protected by @Transactional boundaries to ensure consistency and ACID properties during database writes.
- **Pragmatic Design Approach:**
 - Considering the current scope and controlled dimensions of the project, agile development of explicit, authorial DTOs was chosen. The overhead of *API First* approaches based on code-generation via Swagger/YAML was intentionally avoided, keeping the codebase lean and simple to maintain.

2. High-Resilience Messaging (Apache Kafka)

Complete decoupling between request reception and database persistence is guaranteed by **Apache Kafka**:

- **Consumption Strategy (earliest):**
 - Strictly configured with `spring.kafka.consumer.auto-offset-reset=earliest`.
 - This ensures ultimate fault tolerance: if the consumer microservice experiences an infrastructure crash or scheduled maintenance, no invoice sent by the Frontend is lost. Messages remain securely stored in the broker and are processed sequentially as soon as the worker re-establishes the connection.

3. Hybrid Development Environment (IntelliJ + WSL2 + Docker)

To simulate production conditions locally, I structured an integrated development architecture using **WSL2 (Ubuntu)** and **Docker**:

- **Resource Isolation:** Apache Kafka and Zookeeper run containerized inside Docker Compose, managed natively by the WSL2 Linux kernel.
- **IntelliJ/WSL2 Network Wiring:** IntelliJ IDEA (running on Windows) communicates via local loopback (`localhost:9092`) directly with the containers inside Linux.
- **Credential Externalization (Zero-Trust Security):**
 - **No passwords are exposed** inside the `application.properties` file. The project adopts strict corporate security practices using dynamic injection (`${DB_ORACLE_PASS}`).
 - Locally, secrets are supplied via IDE environment variables or hidden JVM parameters.

4. Cloud Infrastructure, Clustering & DevOps (OCI + Kubernetes)

Deployment and operations utilize the full potential of **Oracle Cloud (OCI)** and modern DevOps practices:

- **Hunting Ampere Instances (Infra Automation):**
 - To bypass global scarcity and high demand for free cloud resources on Oracle, I developed an **autonomous script/bot** integrated with the **OCI CLI**.
 - This bot monitored Oracle's APIs and successfully provisioned **3 Ampere ARM compute instances** within the *Always Free* tier, maximizing CPU and memory limits at zero cost.
- **High-Availability Kubernetes Cluster (K3s):**
 - The 3 virtual machines form a real cluster orchestrated via **K3s**. Selecting K3s was a strategic choice: a lightweight Kubernetes distribution perfectly optimized for ARM architectures, preserving hardware resources while maintaining full **Kubectl** capabilities.
 - **High Availability (HA):** The cluster was configured so that if one of the nodes physically fails, the remaining nodes automatically take over the application replicas, ensuring zero downtime.
- **Automated CI/CD:** The source code hosted on **GitHub** triggers automated pipelines via **GitHub Actions**, which handle image building, testing, Docker artifact generation, and Continuous Deployment (CD) straight into the K3s cluster in the cloud.

5. Hybrid Data Persistence & Secure Cloud

Invoice logging and final file availability follow rigid cloud security standards:

- **Oracle Autonomous Transaction Processing (ATP):**
 - The relational database engine is Oracle's autonomous tier, configured with **mandatory mTLS**.
 - The microservice-to-database communication is secured end-to-end using a **physical SSL Certificate Wallet**, integrated via Java cryptography extensions (oraclepki, osdt_core, osdt_cert).
- **OCI Vault (Production Secrets):** In the cloud production environment, database and Wallet passwords are encrypted and dynamically injected by Oracle's Secret Management Service, maintaining strict compliance with security audits.
- **Hybrid Persistence & Downloads via OCI Object Storage:**
 - In addition to extracting structured data into relational database tables, the Consumer generates a physical .json file and performs an asynchronous upload to a bucket inside **Oracle Object Storage**.

- To guarantee secure downloads for the Frontend, the system generates temporary **Pre-Authenticated Requests (PAR)** URLs. This allows the end-user to download the invoice file directly from the cloud with a pre-programmed expiration window, protecting the link against leaks.
-

6. Front-end Layer & Unified Ingestion (Vanilla JS & Tailwind)

To provide an enterprise-grade control dashboard and simulate a real production environment, a rich client-side application was developed with a focus on raw performance. Adopting a **Vanilla Enterprise architecture (Pure JS + Native CSS)**, it completely eliminates framework overhead:

- **Hybrid Ingestion Engine:** An intelligent tabbed interface that dynamically manages formatted JSON payloads (*pretty-print*). The core asynchronously switches between single payload transmission (*/single*) and mass commercial batch processing (*/batch*) straight to the Producer API on port 8081.
- **Live Feed Traffic Auditing:** A real-time monitor panel that dynamically renders incoming invoices as they hit the messaging grid, shifting visual states to OCI Storage Ready only when cloud persistence is strictly acknowledged.
- **Local Assets Architecture (Zero-CORS/Offline):** Direct deployment of the **JSZip** compression library within the project's internal directory (local assets). This shields the application against strict corporate firewall/CDN blocks and guarantees execution autonomy.
- **Resilient Startup Hydration Engine:** On page load or F5 refresh, an automated lifecycle hook fires a GET request to an isolated history API (*/api/history/all*), designed under **Clean Architecture** and single responsibility principles. The grid is instantly hydrated with streaming records pulled from the relational database, securing visual state persistence.
- **Cascading Hybrid Purge Mechanism (Batch/Single Delete):** Implemented an atomic data obliteration flow governed by a transactional `@DeleteMapping` resource server. The microservice fires a direct query via a Spring Data JPA autogenerated query method (`deleteByNumeroNota`) while simultaneously calling the Oracle Cloud SDK to remove the physical `.json` file from the São Paulo Object Storage. Includes custom fallback handling for 404 (`ObjectNotFound`) exceptions, ensuring system resilience even if cloud sync anomalies occur.
- **Premium Dark Mode User Experience (Custom Modal UX):** Replaced native browser pop-ups (confirm) with a **custom asynchronous dark mode confirmation modal (Blur Backdrop)** engineered using JavaScript Promises, hiding host system paths and setting an elegant corporate layout baseline.

- **Hybrid Download Mechanism (Temporary PAR Encryption):**
 1. **Individual Download:** The action button triggers the Consumer microservice (8082), which leverages the Oracle Cloud SDK to generate a temporary, cryptographically secure **PAR (Pre-Authenticated Request)** link. The JavaScript intercepts the stream and downloads the raw JSON via *Blob Streaming* directly to the user's OS.
 2. **Batch Download (Download All):** Spawns parallel asynchronous requests (Promise.all) to the São Paulo cloud bucket. The front-end fetches the files in the background, compresses all requested invoices into a single .zip archive inside the browser's RAM, and delivers a single packaged download, optimizing network transmission channels.
-

7. Infrastructure Topology & Cluster Governance (Kubernetes / K3s)

To ensure strict process isolation and simulate a real distributed production environment under cloud budget constraints (*Cloud Cost Optimization*), the layout is deployed as a **3-Node Hybrid Cluster (1 Master + 2 Workers)** inside Oracle Cloud Infrastructure (OCI):

- **Master Node (Control Plane):** Centralizes cluster infrastructure orchestration and exclusively hosts the messaging backbone powered by **Single-Node Apache Kafka (Integrated KRaft Mode)**. This node operates isolation workflows managing metadata consensus and topic routing.
- **Worker Nodes (Data Plane):** Independent computing instances purely dedicated to running application microservices. The **Producer API (8081)** is isolated on Worker 1, whereas the **Consumer engine (8082)** executes on Worker 2.
- **Distributed Internal Routing:** All network traffic routes transparently through native Kubernetes cluster networking. Microservices running on separate worker nodes dispatch messaging payloads in the background directly to the Kafka cluster inside the master node, accurately replicating corporate cross-server network behaviors.

8. Testing

To close the project lifecycle, visual validation was structured into two layers:

Frontend / API Tooling: Users can utilize pre-configured **Postman** collections to trigger batch loads or complex raw JSON payloads into the API, testing the system's throughput thresholds.

9. Observability, Auditing & Platform Governance (Kafdrop & Rancher)

To ensure governance and real-time monitoring of the entire distributed infrastructure, two market solutions were integrated:

- **Kafdrop (Web UI for Apache Kafka):** Implemented as a lightweight container wired into the messaging grid on port 9000. It enables instant visual auditing of topics, consumer group lag monitoring for faturamento-group, and direct real-time payload inspection, eliminating the need for CLI tools to validate message flows.
- **Rancher (Kubernetes Management & Governance):** Utilized as the command center to manage the K3s cluster distributed across the 3 cloud ARM Ampere instances. Rancher provides a robust enterprise dashboard to monitor microservices Pod health, manage persistent volumes, control load balancing, and audit hardware resources in real time, perfectly simulating a production-grade operations room of a large corporation.

10. Apex

User Interface (Oracle APEX): Development of an agile control and monitoring dashboard built in **Oracle APEX (Low-Code)**, directly integrated with the Autonomous Database tables. This allows users to check invoicing statuses, view structured logs, and click the link generated by Object Storage to execute a **secure download** of the physical JSON invoice file seamlessly.